

CPU Scheduling

CPU scheduling is an Operating System mechanism that decides which process gets the CPU next. It is mainly used in multiprogramming systems, where many processes are kept in memory and the CPU switches between them to improve utilization, responsiveness, and throughput.

For placements, CPU scheduling is important because questions are often asked in two forms:

1. Conceptual questions: definitions, advantages, disadvantages, starvation, aging, preemption.
 2. Numerical problems: Gantt chart, waiting time, turnaround time, response time, completion time.
-

1. Basic Terms

Process

A process is a program in execution. Each process may need CPU time, I/O time, memory, and other resources.

Burst Time

Burst time, also called CPU burst time, is the amount of CPU time required by a process to complete its execution.

Example:

Process	Burst Time
P1	5 ms
P2	3 ms

Here, P1 needs 5 ms of CPU time, and P2 needs 3 ms of CPU time.

Arrival Time

Arrival time is the time at which a process enters the ready queue.

Example:

Process	Arrival Time
P1	0 ms
P2	2 ms

P1 is available from time 0, while P2 becomes available at time 2.

Ready Queue

The ready queue contains processes that are ready to execute and are waiting for CPU allocation.

Completion Time

Completion time is the time at which a process finishes execution.

Turnaround Time

Turnaround time is the total time taken by a process from arrival to completion.

Formula:

$$\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$$

Waiting Time

Waiting time is the total time a process spends waiting in the ready queue.

Formula:

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

For preemptive scheduling, waiting time includes all waiting intervals, not just one continuous wait.

Response Time

Response time is the time from process arrival until the process gets the CPU for the first time.

Formula:

$$\text{Response Time} = \text{First CPU Start Time} - \text{Arrival Time}$$

Response time is especially important in interactive systems because users care about how quickly the system starts responding.

Throughput

Throughput is the number of processes completed per unit time.

CPU Utilization

CPU utilization is the percentage of time the CPU is busy doing useful work.

Context Switch

A context switch happens when the CPU switches from one process to another. The OS saves the current process state and loads the next process state.

Context switching is necessary, but it adds overhead because no actual process execution happens during the switch.

2. Goals of CPU Scheduling

A good CPU scheduling algorithm tries to:

- Maximize CPU utilization.
- Maximize throughput.
- Minimize waiting time.
- Minimize turnaround time.
- Minimize response time.
- Provide fairness among processes.
- Avoid starvation.
- Balance CPU-bound and I/O-bound processes.

No single algorithm is best for every situation. Different algorithms optimize different goals.

3. Preemptive vs Non-Preemptive Scheduling

Non-Preemptive Scheduling

In non-preemptive scheduling, once a process gets the CPU, it keeps the CPU until it completes or voluntarily moves to the waiting state.

Examples:

- FCFS
- Non-preemptive SJF
- Non-preemptive Priority Scheduling

Advantages:

- Simple to implement.
- Less context switching overhead.
- Predictable execution.

Disadvantages:

- Poor response time.
- A long process can block shorter processes.
- Not ideal for time-sharing systems.

Preemptive Scheduling

In preemptive scheduling, the OS can forcibly take the CPU away from a process and give it to another process.

Examples:

- SRTF
- Round Robin
- Preemptive Priority Scheduling

Advantages:

- Better response time.
- Useful for interactive systems.
- Prevents one process from monopolizing the CPU.

Disadvantages:

- More context switching overhead.
- More complex implementation.
- Can cause starvation in some algorithms.

4. First Come First Serve

FCFS stands for First Come First Serve.

In FCFS scheduling, the process that arrives first gets the CPU first. It works like a normal queue.

Type

FCFS is non-preemptive.

Rule

Processes are executed in increasing order of arrival time.

If two processes have the same arrival time, the tie is usually resolved by process ID order or input order, depending on the question.

Example

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Execution order:

P1 -> P2 -> P3 -> P4

Gantt chart:

```
0    5    8    16    22
| P1 | P2 | P3 | P4 |
```

Completion times:

Process	Completion Time
P1	5
P2	8
P3	16
P4	22

Turnaround time:

$TAT = \text{Completion Time} - \text{Arrival Time}$

Process	CT	AT	TAT
P1	5	0	5
P2	8	1	7
P3	16	2	14
P4	22	3	19

Waiting time:

$WT = \text{Turnaround Time} - \text{Burst Time}$

Process	TAT	BT	WT
P1	5	5	0
P2	7	3	4
P3	14	8	6
P4	19	6	13

Average waiting time:

$(0 + 4 + 6 + 13) / 4 = 5.75$

Average turnaround time:

$$(5 + 7 + 14 + 19) / 4 = 11.25$$

Advantages

- Very simple.
- Easy to implement using a FIFO queue.
- No starvation because every process eventually gets CPU.
- Low scheduling overhead.

Disadvantages

- Poor average waiting time.
- Poor response time for short processes behind long processes.
- Not suitable for interactive systems.
- Causes convoy effect.

Convoy Effect

The convoy effect occurs when a long process holds the CPU and many short processes wait behind it.

This reduces system performance because short jobs that could finish quickly are delayed by one long job.

Placement line:

FCFS suffers from convoy effect.

5. Shortest Job First

SJF stands for Shortest Job First.

In SJF scheduling, the process with the smallest burst time is executed first.

Type

SJF is usually non-preemptive unless stated otherwise.

Its preemptive version is called SRTF, which is covered later.

Rule

Among the processes available in the ready queue, select the process with the smallest burst time.

Important: SJF does not always choose the globally shortest process from the full table. It chooses the shortest process among the processes that have already arrived.

Example

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

At time 0, only P1 is available. Since SJF is non-preemptive, P1 starts and runs completely.

Gantt chart:

```

0    7    8    12   16
| P1 | P3 | P2 | P4 |

```

Explanation:

- At time 0, only P1 is available, so P1 runs from 0 to 7.
- At time 7, P2, P3, and P4 are available.
- P3 has the shortest burst time, so P3 runs next.
- P2 and P4 both have burst time 4. Tie is resolved by earlier arrival, so P2 runs before P4.

Completion times:

Process	Completion Time
P1	7
P3	8
P2	12
P4	16

Turnaround and waiting times:

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	7	7	7	0
P2	2	4	12	10	6
P3	4	1	8	4	3
P4	5	4	16	11	7

Average waiting time:

$$(0 + 6 + 3 + 7) / 4 = 4$$

Average turnaround time:

$$(7 + 10 + 4 + 11) / 4 = 8$$

Advantages

- Gives minimum average waiting time if all processes are available at the beginning.
- Short jobs finish quickly.
- Better than FCFS for batch systems.

Disadvantages

- Difficult to know the exact CPU burst time in advance.
- Long processes may starve if short processes keep arriving.
- Not ideal for interactive systems in non-preemptive form.

Starvation in SJF

Starvation can occur because long processes may keep waiting while shorter processes continue to arrive.

Example:

If a process needs 50 ms and many processes of 1 ms, 2 ms, and 3 ms keep arriving, the 50 ms process may wait for a very long time.

6. Shortest Remaining Time First

SRTF stands for Shortest Remaining Time First.

SRTF is the preemptive version of SJF.

Type

SRTF is preemptive.

Rule

At every scheduling decision, choose the process with the smallest remaining burst time.

If a new process arrives with a burst time smaller than the remaining time of the currently running process, the current process is preempted.

Example

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	2
P4	3	1

Step-by-step:

- At time 0, only P1 is available. P1 starts.
- At time 1, P2 arrives. P1 has 7 ms remaining, P2 needs 4 ms. P2 preempts P1.
- At time 2, P3 arrives. P2 has 3 ms remaining, P3 needs 2 ms. P3 preempts P2.
- At time 3, P4 arrives. P3 has 1 ms remaining, P4 needs 1 ms. Tie may be resolved by continuing P3 or by arrival order. We continue P3.
- P3 finishes at time 4.
- P4 runs from 4 to 5.
- P2 runs from 5 to 8.
- P1 runs from 8 to 15.

Gantt chart:

```

0   1   2   4   5   8   15
| P1 | P2 | P3 | P4 | P2 | P1 |

```

Completion times:

Process	Completion Time
P1	15
P2	8
P3	4
P4	5

Turnaround and waiting times:

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	8	15	15	7
P2	1	4	8	7	3
P3	2	2	4	2	0
P4	3	1	5	2	1

Average waiting time:

$$(7 + 3 + 0 + 1) / 4 = 2.75$$

Average turnaround time:

$$(15 + 7 + 2 + 2) / 4 = 6.5$$

Response times:

Process	First Start	Arrival Time	Response Time
P1	0	0	0
P2	1	1	0
P3	2	2	0
P4	4	3	1

Advantages

- Usually gives very low average waiting time.
- Short processes get quick service.
- Better response than non-preemptive SJF.

Disadvantages

- More context switching overhead.
- Long processes can starve.
- Requires continuous tracking of remaining times.
- Exact CPU burst time is hard to know in real systems.

Placement Trap

In SRTF, always compare the new process burst time with the remaining time of the currently running process, not with its original burst time.

7. Round Robin

Round Robin is a preemptive CPU scheduling algorithm designed mainly for time-sharing systems.

Each process gets the CPU for a fixed time slice called a time quantum. If the process does not finish within that time quantum, it is preempted and placed at the end of the ready queue.

Type

Round Robin is preemptive.

Rule

Each process gets at most one time quantum per turn.

If it finishes before the time quantum expires, it leaves the system.

If it does not finish, it goes to the back of the ready queue.

Time Quantum

Time quantum is the maximum amount of CPU time a process can use in one turn.

Choosing the time quantum is very important.

If time quantum is too small:

- Too many context switches.
- More overhead.
- CPU time is wasted in switching.

If time quantum is too large:

- Round Robin behaves like FCFS.
- Response time becomes worse.

Ideal time quantum:

- Large compared to context switch time.
- Small enough to provide good response time.

Example

Assume time quantum = 2 ms.

Process	Arrival Time	Burst Time
P1	0	5
P2	1	4
P3	2	2
P4	3	1

Gantt chart:

```
0    2    4    6    7    9    11    12
| P1 | P2 | P3 | P4 | P1 | P2 | P1 |
```

Explanation:

- P1 runs from 0 to 2. Remaining time = 3.

- P2 runs from 2 to 4. Remaining time = 2.
- P3 runs from 4 to 6. It finishes.
- P4 runs from 6 to 7. It finishes.
- P1 runs from 7 to 9. Remaining time = 1.
- P2 runs from 9 to 11. It finishes.
- P1 runs from 11 to 12. It finishes.

Completion times:

Process	Completion Time
P1	12
P2	11
P3	6
P4	7

Turnaround and waiting times:

Process	AT	BT	CT	TAT = CT - AT	WT = TAT - BT
P1	0	5	12	12	7
P2	1	4	11	10	6
P3	2	2	6	4	2
P4	3	1	7	4	3

Average waiting time:

$$(7 + 6 + 2 + 3) / 4 = 4.5$$

Average turnaround time:

$$(12 + 10 + 4 + 4) / 4 = 7.5$$

Response times:

Process	First Start	Arrival Time	Response Time
P1	0	0	0
P2	2	1	1
P3	4	2	2
P4	6	3	3

Advantages

- Fair algorithm because every process gets CPU regularly.
- Good response time.
- Suitable for time-sharing and interactive systems.
- No starvation if all processes eventually get turns.

Disadvantages

- Performance depends heavily on time quantum.
- More context switching than FCFS and SJF.
- Average waiting time may be high.
- Not always optimal for turnaround time.

Important Placement Points

- Round Robin is preemptive.
 - It is fair.
 - It is commonly used in time-sharing systems.
 - Very large time quantum makes it similar to FCFS.
 - Very small time quantum causes too many context switches.
-

8. Priority Scheduling

In Priority Scheduling, each process is assigned a priority, and the CPU is allocated to the process with the highest priority.

Priority may be based on:

- Memory requirements.
- Time limits.
- Importance of process.
- Process type.
- User type.
- Internal OS criteria.

Type

Priority scheduling can be:

- Non-preemptive.
- Preemptive.

Priority Number Confusion

Different textbooks use different conventions.

In many OS textbooks:

Lower priority number = Higher priority

Example:

Priority 1 is higher than priority 5.

But in some systems:

Higher priority number = Higher priority

Always check the question statement.

Non-Preemptive Priority Scheduling

Once a process starts running, it continues until completion, even if a higher-priority process arrives later.

Example:

Assume lower number means higher priority.

Process	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	4	1
P3	2	2	4
P4	3	3	2

At time 0, only P1 is available, so P1 starts and completes.

At time 6, P2, P3, and P4 are available. Priority order is P2, P4, P3.

Gantt chart:

0 6 10 13 15
| P1 | P2 | P4 | P3 |

Preemptive Priority Scheduling

If a new process arrives with higher priority than the currently running process, the current process is preempted.

Example:

Assume lower number means higher priority.

Process	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	4	1
P3	2	2	4
P4	3	3	2

Step-by-step:

- At time 0, P1 starts.
- At time 1, P2 arrives with higher priority than P1. P1 is preempted.
- P2 runs from 1 to 5 and finishes.
- At time 5, P1, P3, and P4 are available. P4 has highest priority among them.
- P4 runs from 5 to 8.
- P1 runs from 8 to 13.
- P3 runs from 13 to 15.

Gantt chart:

```
0      1      5      8      13     15
| P1 | P2 | P4 | P1 | P3 |
```

Advantages

- Important processes can be executed first.
- Useful in real-time systems.
- Flexible because priority can represent many factors.

Disadvantages

- Can cause starvation of low-priority processes.
- Requires priority assignment.
- If priorities are poorly chosen, performance becomes poor.
- Preemptive priority scheduling may have high context switching overhead.

Starvation in Priority Scheduling

Starvation is a major problem in priority scheduling.

If high-priority processes keep arriving, low-priority processes may never get the CPU.

Example:

A low-priority process P10 is waiting. If new high-priority processes keep entering the ready queue, P10 keeps getting delayed indefinitely.

9. Starvation

Starvation, also called indefinite blocking, occurs when a process waits for a very long time because the scheduling algorithm keeps selecting other processes.

Starvation does not mean deadlock.

In deadlock, processes are blocked forever because each process is waiting for a resource held by another process.

In starvation, the process is ready to run but is repeatedly ignored by the scheduler.

Algorithms That Can Cause Starvation

Starvation can occur in:

- SJF.
- SRTF.
- Priority Scheduling.

Starvation generally does not occur in:

- FCFS, because processes are served in arrival order.
- Round Robin, because every process gets a turn.

Example of Starvation

Suppose a long process needs 100 ms of CPU time.

If short processes of 1 ms or 2 ms keep arriving, SJF or SRTF may keep selecting those shorter processes.

The long process keeps waiting.

This is starvation.

10. Aging

Aging is a technique used to prevent starvation.

In aging, the priority of a waiting process is gradually increased as it spends more time in the ready queue.

Eventually, even a low-priority process becomes high priority enough to get the CPU.

Example

Assume lower priority number means higher priority.

Suppose P1 has priority 10 and has been waiting for a long time.

The OS may gradually improve its priority:

```
10 -> 9 -> 8 -> 7 -> ... -> 1
```

After enough waiting, P1 becomes high priority and gets CPU time.

Why Aging Works

Aging prevents indefinite waiting.

It ensures that no process remains ignored forever.

Placement Line

Aging is used to solve starvation by gradually increasing the priority of waiting processes.

11. Waiting Time, Turnaround Time, and Response Time

These three metrics are extremely important in CPU scheduling numericals.

Completion Time

Completion time is when the process finishes.

```
CT = Finish Time
```

Turnaround Time

Turnaround time is the total time spent by the process in the system.

```
TAT = CT - AT
```

Where:

- CT = Completion Time
- AT = Arrival Time

Waiting Time

Waiting time is the total time spent waiting in the ready queue.

```
WT = TAT - BT
```

Where:

- TAT = Turnaround Time
- BT = Burst Time

Another form:

$$WT = CT - AT - BT$$

Response Time

Response time is the time between arrival and first CPU allocation.

$$RT = \text{First Start Time} - AT$$

Response time is not the same as waiting time in preemptive algorithms.

Important Difference Between Waiting Time and Response Time

For non-preemptive algorithms, waiting time and response time are often the same because once a process starts, it runs until completion.

For preemptive algorithms, waiting time and response time are usually different.

Example:

A process arrives at time 0, starts at time 2, gets preempted, waits again, and completes at time 10.

Its response time is:

$$2 - 0 = 2$$

But its waiting time includes all waiting intervals before and after preemption.

12. How to Solve CPU Scheduling Numericals

Use this method in exams and interviews:

1. Write the process table clearly.
2. Identify the scheduling algorithm.
3. Check whether it is preemptive or non-preemptive.
4. Draw the Gantt chart.
5. Find completion time from the Gantt chart.
6. Calculate turnaround time using $CT - AT$.
7. Calculate waiting time using $TAT - BT$.

8. Calculate response time using **First Start Time - AT**.

9. Find averages if asked.

Common Mistakes

- Using burst time instead of remaining time in SRTF.
- Forgetting arrival times.
- Assuming SJF chooses the shortest process from the whole table before it has arrived.
- Confusing waiting time with response time.
- Forgetting that Round Robin puts unfinished processes at the end of the ready queue.
- Not checking whether lower priority number means higher priority.
- Ignoring idle CPU time when no process has arrived yet.

13. CPU Idle Time

Sometimes no process is available at a certain time. In that case, the CPU remains idle.

Example:

Process	Arrival Time	Burst Time
P1	3	4
P2	5	2

From time 0 to 3, no process has arrived.

Gantt chart:

```

0       3       7       9
| Idle | P1  | P2  |

```

Do not start P1 at time 0 because it arrives at time 3.

14. Comparison Table

Algorithm	Preemptive?	Selection Rule	Starvation?	Main Advantage	Main Disadvantage
FCFS	No	Earliest arrival first	No	Simple and fair by arrival order	Convoy effect
SJF	No	Smallest burst time among arrived processes	Yes	Low average waiting time	Needs burst time prediction
SRTF	Yes	Smallest remaining time	Yes	Very good average waiting	More context switching

Algorithm	Preemptive?	Selection Rule	Starvation?	Main Advantage	Main Disadvantage
				time	
Round Robin	Yes	Fixed time quantum cyclic order	No	Good response and fairness	Depends on time quantum
Priority	Both	Highest priority first	Yes	Important processes run first	Low-priority starvation

15. Interview-Ready Answers

What is CPU scheduling?

CPU scheduling is the OS mechanism that selects which ready process should get the CPU next.

Why is CPU scheduling needed?

It is needed to improve CPU utilization, throughput, response time, turnaround time, and fairness among processes.

Which scheduling algorithm gives minimum average waiting time?

SJF gives minimum average waiting time when all processes are available at the beginning and burst times are known.

SRTF is the preemptive version and often gives even better average waiting time when processes arrive at different times.

Which scheduling algorithm is best for time-sharing systems?

Round Robin is commonly used for time-sharing systems because each process gets CPU time regularly.

What happens if the Round Robin time quantum is too small?

There will be too many context switches, causing high overhead.

What happens if the Round Robin time quantum is too large?

Round Robin behaves like FCFS.

What is starvation?

Starvation occurs when a process waits indefinitely because other processes keep getting selected by the scheduler.

How can starvation be solved?

Starvation can be solved using aging, where the priority of a waiting process is gradually increased over time.

What is the difference between SJF and SRTF?

SJF is non-preemptive and selects the process with the shortest burst time among arrived processes.

SRTF is preemptive and always selects the process with the shortest remaining time.

What is the convoy effect?

The convoy effect occurs in FCFS when many short processes wait behind a long process, increasing average waiting time.

What is the difference between turnaround time and waiting time?

Turnaround time is the total time from arrival to completion.

Waiting time is the time spent waiting in the ready queue.

$$TAT = CT - AT$$

$$WT = TAT - BT$$

What is the difference between response time and waiting time?

Response time is the time from arrival until the first CPU allocation.

Waiting time is the total time spent waiting in the ready queue.

In preemptive scheduling, they are usually different.

16. Quick Formula Sheet

Completion Time (CT) = Time at which process finishes

Turnaround Time (TAT) = Completion Time - Arrival Time

Waiting Time (WT) = Turnaround Time - Burst Time

Waiting Time (WT) = Completion Time - Arrival Time - Burst Time

Response Time (RT) = First CPU Start Time - Arrival Time

Average Waiting Time = Sum of waiting times / Number of processes

Average Turnaround Time = Sum of turnaround times / Number of processes

Average Response Time = Sum of response times / Number of processes

17. Final Revision Notes

- FCFS is simple but suffers from convoy effect.
- SJF minimizes average waiting time but can cause starvation.
- SRTF is preemptive SJF and uses remaining time.
- Round Robin is fair and good for time-sharing systems.
- Priority scheduling can be preemptive or non-preemptive.
- Starvation means a ready process waits indefinitely.
- Aging solves starvation by increasing priority over time.
- Turnaround time is total time in system.
- Waiting time is total time in ready queue.
- Response time is time until first CPU response.
- Always draw the Gantt chart first in numerical problems.